

Experiment 1: Decision Tree Classification

Aim

To implement and evaluate a Decision Tree classifier on the Iris dataset using Python and Scikit-learn, and to analyze its classification performance.

(a) Dataset Loading, Pre-processing and Splitting

The **Iris dataset** contains 150 observations belonging to three flower classes: *setosa*, *versicolor*, and *virginica*. Each observation has four numerical features:

- Sepal length
- Sepal width
- Petal length
- Petal width

The dataset does not contain missing values or categorical input features, so no missing-value imputation or encoding is necessary. The data is divided into **80% training** and **20% testing data**, using stratified sampling.

Python Code

```
import pandas as pd

from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier, export_text

from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Load Iris dataset

iris = load_iris(as_frame=True)

X = iris.data

y = iris.target
```

```
# Create a complete dataframe for displaying data
```

```
df = X.copy()
```

```
df["target"] = y
```

```
# Display first 5 rows
```

```
print("First 5 rows:")
```

```
print(df.head())
```

```
# Display data types
```

```
print("\nData types:")
```

```
print(df.dtypes)
```

```
# Check missing values
```

```
print("\nMissing values:")
```

```
print(df.isnull().sum())
```

```
# Train-test split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X,
```

```
    y,
```

```
    test_size=0.20,
```

```
    random_state=42,
```

```
    stratify=y
```

```
)
```

```
print("\nTraining samples:", len(X_train))
```

```
print("Testing samples:", len(X_test))
```

First 5 Rows

Sepal Length	Sepal Width	Petal Length	Petal Width	Target
--------------	-------------	--------------	-------------	--------

5.1	3.5	1.4	0.2	0
4.9	3.0	1.4	0.2	0
4.7	3.2	1.3	0.2	0
4.6	3.1	1.5	0.2	0
5.0	3.6	1.4	0.2	0

Data Types

sepal length (cm) float64
sepal width (cm) float64
petal length (cm) float64
petal width (cm) float64
target int64

There are **no missing values**, and all four input features are numerical.

(b) Building the Decision Tree

A Decision Tree classifier is constructed using the **Gini Index** as the splitting criterion.

Python Code

```
# Create Decision Tree using Gini Index
```

```
dt_model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
```

```
# Train the model
```

```
dt_model.fit(X_train, y_train)
```

```
# Print tree structure
```

```
tree_rules = export_text(
    dt_model,
```

```
feature_names=list(X.columns)
)
```

```
print("Decision Tree Structure:")
print(tree_rules)
```

Obtained Tree Structure

```
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) > 2.45
|   |--- petal width (cm) <= 1.65
|       |--- petal length (cm) <= 4.95
|       |   |--- class: 1
|       |   |--- petal length (cm) > 4.95
|       |       |--- sepal length (cm) <= 6.15
|       |       |   |--- sepal width (cm) <= 2.45
|       |       |       |--- class: 2
...

```

Root Node

The **root node** is **petal length <= 2.45 cm**.

This is selected because it provides the greatest reduction in impurity among the candidate features and thresholds. For the training dataset, the initial Gini impurity is approximately **0.667**. The split at petal length = 2.45 cm produces a weighted impurity of approximately **0.333**, giving a Gini impurity reduction of approximately **0.333**.

The feature importance values also show that **petal length** and **petal width** are the dominant features:

Feature	Importance
Sepal length	0.0063
Sepal width	0.0292
Petal length	0.5586

Petal width	0.4060
--------------------	---------------

Thus, petal length is a logical root feature because it most effectively separates the Iris classes.

(c) Model Evaluation

Python Code

```
# Make predictions
```

```
y_pred = dt_model.predict(X_test)
```

```
# Accuracy
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
print("Accuracy:", accuracy)
```

```
# Confusion Matrix
```

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("\nConfusion Matrix:")
```

```
print(cm)
```

```
# Classification Report
```

```
print("\nClassification Report:")
```

```
print(
```

```
    classification_report(
```

```
        y_test,
```

```
        y_pred,
```

```
        target_names=iris.target_names
```

```
    )
```

```
)
```

Accuracy

The model achieved an accuracy of:

Accuracy = 93.33%

Therefore, the Decision Tree correctly classified **28 out of 30 test observations**.

Confusion Matrix

```
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]
```

Interpreting the matrix:

- All **10 Setosa** samples were correctly classified.
- **9 of 10 Versicolor** samples were correctly classified.
- **9 of 10 Virginica** samples were correctly classified.
- Two samples were misclassified.

Classification Report

Class	Precision	Recall	F1-score
Setosa	1.00	1.00	1.00
Versicolor	0.90	0.90	0.90
Virginica	0.90	0.90	0.90
Macro Average	0.93	0.93	0.93
Weighted Average	0.93	0.93	0.93

Misclassified Instances

Two test samples were incorrectly classified:

Dataset Index	Actual Class	Predicted Class	Petal Length	Petal Width
134	Virginica	Versicolor	5.6	1.4
77	Versicolor	Virginica	5.0	1.7

Performance Comment

The Decision Tree performs very well, achieving **93.33% test accuracy**. Setosa is completely separated from the other classes because its petal measurements are substantially smaller. The remaining errors occur between Versicolor and Virginica because these two species have overlapping petal characteristics.

The two errors therefore illustrate an important limitation of Decision Trees: when class distributions overlap, a simple series of threshold-based decisions may not perfectly distinguish the classes.

Experiment 2: Reinforcement Learning – Q-Learning

Aim

To implement the Q-Learning algorithm for a **4×4 Grid World**, learn an optimal policy through repeated interaction with the environment, and observe the evolution of Q-values.

(a) Environment Definition

The Grid World consists of **16 cells arranged in a 4×4 grid**.

We use:

- Starting state: (0,0)
- Goal state: (3,3)
- Obstacles: (1,1) and (2,2)
- Actions: **Up, Down, Left, Right**

Reward Structure

Event	Reward
Reaching goal	+10
Normal movement	−1
Moving into obstacle	−5
Attempting to move outside grid	−1

The negative step reward encourages the agent to reach the goal using a short path, while the obstacle penalty discourages collisions.

Hyperparameters

Hyperparameter	Value
Learning rate α	0.10
Discount factor	0.90

γ	
Initial ϵ	0.90
Minimum ϵ	0.05
Epsilon decay	0.98

The high initial epsilon encourages exploration. As training progresses, epsilon decreases, so the agent increasingly exploits the learned Q-values.

Q-Learning Formula

The Q-value is updated using:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

where:

- s = current state
- a = selected action
- R = reward
- s' = next state
- α = learning rate
- γ = discount factor

Python Implementation

```
import numpy as np
import matplotlib.pyplot as plt

# -----
# 1. Define the Grid World
# -----

grid_size = 4
```



```
start = (0, 0)
```

```
goal = (3, 3)
```

```
# Obstacles
```

```
obstacles = {
```

```
    (1, 1),
```

```
    (2, 2)
```

```
}
```

```
actions = ["Up", "Down", "Left", "Right"]
```

```
moves = {
```

```
    "Up": (-1, 0),
```

```
    "Down": (1, 0),
```

```
    "Left": (0, -1),
```

```
    "Right": (0, 1)
```

```
}
```

```
# Valid states
```

```
states = [
```

```
    (r, c)
```

```
    for r in range(grid_size)
```

```
    for c in range(grid_size)
```

```
    if (r, c) not in obstacles
```

```
]
```

```
state_index = {
```

```
    state: i
```

```

        for i, state in enumerate(states)
    }

# -----
# 2. Initialize Q-table
# -----

Q = np.zeros((len(states), len(actions)))

alpha = 0.10
gamma = 0.90
epsilon = 0.90
epsilon_decay = 0.98
epsilon_min = 0.05

rng = np.random.default_rng(42)

# -----
# 3. Environment step
# -----

def step(state, action):

    r, c = state
    dr, dc = moves[action]

    next_state = (r + dr, c + dc)

```

```

# Outside grid
if (
    next_state[0] < 0
    or next_state[0] >= grid_size
    or next_state[1] < 0
    or next_state[1] >= grid_size
):
    return state, -1, False

# Obstacle
if next_state in obstacles:
    return state, -5, False

# Goal
if next_state == goal:
    return next_state, 10, True

# Normal movement
return next_state, -1, False

# -----
# 4. Train the Q-learning agent
# -----

episodes = 500
max_steps = 100

cumulative_rewards = []

```

```
# Store Q-values at required episodes
saved_q_values = {}

for episode in range(1, episodes + 1):

    state = start
    total_reward = 0

    for step_number in range(max_steps):

        state_id = state_index[state]

        # Epsilon-greedy action selection
        if rng.random() < epsilon:
            action_id = rng.integers(len(actions))
        else:
            action_id = np.argmax(Q[state_id])

        action = actions[action_id]

        next_state, reward, done = step(state, action)

        next_state_id = state_index[next_state]

        # Q-learning update
        if done:
            target = reward
        else:
```

```

        target = (
            reward
            + gamma * np.max(Q[next_state_id])
        )

    Q[state_id, action_id] += (
        alpha
        * (target - Q[state_id, action_id])
    )

    state = next_state
    total_reward += reward

    if done:
        break

cumulative_rewards.append(total_reward)

# Save Q-values at Episodes 1, 50 and 100
if episode in [1, 50, 100]:

    saved_q_values[episode] = {
        "(0,0)": Q[state_index[(0,0)]].copy(),
        "(1,0)": Q[state_index[(1,0)]].copy()
    }

# Reduce exploration
epsilon = max(
    epsilon_min,

```

```

        epsilon * epsilon_decay
    )

# -----
# 5. Display Q-values
# -----

for episode, values in saved_q_values.items():

    print("\nEpisode", episode)

    for state, q_values in values.items():

        print(
            state,
            dict(zip(actions, q_values))
        )

```

(b) Q-Table Evolution

Two representative states are selected:

- State (0,0) — starting state
- State (1,0) — state immediately below the starting point

The following values were obtained during the run.

State (0,0)

Episode	Up	Down	Left	Right
1	-0.419	-0.271	-0.190	-0.271
50	-2.571	-2.226	-2.482	-2.331
100	-2.084	1.030	-2.241	-2.383

At Episode 1, the Q-values are close to zero because the table has just started learning.

By Episode 50, the agent has accumulated more information about the environment, and the Q-values become more negative because repeated movement incurs the step penalty.

At Episode 100, the Q-value for **Down becomes positive and largest**, showing that the agent has learned that moving downward from (0,0) is a good first action.

State (1,0)

Episode	Up	Down	Left	Right
1	-0.199	-0.190	0.000	-0.950
50	-2.072	-1.034	-1.917	-5.455
100	-1.982	2.670	-1.885	-5.247

The Q-value for **Down** becomes the largest value by Episode 100. This indicates that the agent has learned to continue downward from this state instead of moving toward the obstacle at (1,1).

The exact numerical values can vary slightly between runs because Q-Learning uses random exploration.

(c) Final Learned Policy

After training, the optimal action at each state is obtained using:

```
# Extract final policy
```

```
final_policy = {}
```

```
for state in states:
```

```
    state_id = state_index[state]
```

```
    best_action = actions[  
        np.argmax(Q[state_id])
```

```
    ]
```

```
final_policy[state] = best_action
```

```
print("\nFinal Learned Policy:")
```

```
for state, action in final_policy.items():
```

```
    print(state, "->", action)
```

A compact representation of the learned policy is:

Grid Position	Learned Action
(0,0)	↓ Down
(0,1)	→ Right
(0,2)	→ Right
(0,3)	↓ Down
(1,0)	↓ Down
(1,2)	→ Right
(1,3)	↓ Down
(2,0)	→ Right
(2,1)	↓ Down
(2,3)	↓ Down
(3,0)	→ Right
(3,1)	→ Right
(3,2)	→ Right
(3,3)	Goal

The obstacle cells (1,1) and (2,2) have no learned action because they are not valid states.

Policy Interpretation

Starting at (0,0), the agent follows approximately:

Start

↓
(1,0)
↓
(2,0)
→
(2,1)
↓
(3,1)
→
(3,2)
→
Goal (3,3)

The learned policy avoids the obstacle cells and progressively moves toward the goal.

Cumulative Reward Plot

The reward obtained in every episode can be plotted using:

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(  
    cumulative_rewards,  
    label="Cumulative Reward"  
)
```

```
plt.xlabel("Episode")
```

```
plt.ylabel("Reward")
```

```
plt.title("Q-Learning: Cumulative Reward per Episode")
```

```
plt.grid(True)
```

```
plt.legend()
```

```
plt.show()
```

For a clearer view of convergence, a moving average can also be plotted:

```
window = 20
```

```
moving_average = np.convolve(  
    cumulative_rewards,  
    np.ones(window) / window,  
    mode="valid"  
)
```

```
plt.figure(figsize=(10, 5))
```

```
plt.plot(  
    moving_average,  
    label="20-Episode Moving Average"  
)
```

```
plt.xlabel("Episode")  
plt.ylabel("Average Reward")  
plt.title("Q-Learning Convergence")  
plt.grid(True)  
plt.legend()
```

```
plt.show()
```

Convergence Behaviour

At the beginning of training, the agent explores the environment extensively because epsilon is high. Consequently, rewards fluctuate significantly and can become highly negative because the agent repeatedly takes unnecessary steps or encounters obstacles.

As training continues:

1. The Q-values become more informative.

2. The agent learns to avoid obstacle states.
3. Epsilon gradually decreases, reducing random exploration.
4. The agent increasingly follows high-value actions.
5. The cumulative reward becomes more stable.

Near the end of training, the reward typically stabilizes around the value expected from taking a short path to the goal. In the implemented experiment, the last several episodes obtain positive rewards around **4-5**, indicating that the agent has learned a short route while still paying the step penalty.